



SHAHEED ZULFIKAR ALI BHUTTO

INSTITUTE OF SCIENCE AND TECHNOLOGY

Software Design Specification (SDS)

TravelGuide

Tour Planning Android Application

Version 1.0 Approved

Students: Munir Ahmed (2212138) & Meer Jaan (2212133)

Project Supervisor: Mr. Sufyan Haqqi

Organization: Shaheed Zulfiqar Ali Bhutto Institute of Science & Technology

Date: 16 March, 2026

Table of Contents

1. Introduction.....	1
1.1 Objective of this Document	1
1.2 Project Development Scope	1
1.3 Definitions, Acronyms, and Abbreviations.....	1
1.4 References.....	2
1.5 Overview of Document.....	2
2. System Architecture Description	3
2.1 Section Overview	3
2.2 General Constraints.....	3
2.3 Data Design.....	3
2.4 Program Structure	4
2.5 Alternatives Considered.....	5
3. Detailed Description of Components.....	5
3.1 Section Overview	5
3.2 User Authentication Component.....	6
3.3 Hotel Module Component.....	6
3.4 Restaurant and Food Module Component	7
3.5 Transport Module Component	7
3.6 Tourist Attractions Module Component	8
3.7 Budget Estimation Component.....	8
3.8 Admin Module Component	9
4. User Interface Design	9
4.1 Section Overview	9
4.2 Interface Design Rules.....	9
4.3 GUI Components	10
4.4 Detailed Description	10
5. Reuse and Relationships to Other Products	11
6. Design Decisions and Tradeoffs	12
7. Pseudocode for Components.....	13
8. Appendices.....	13

1. Introduction

1.1 Objective of this Document

Software Design Description (SDS) provides an overview of TravelGuide system design and conditions required to implement TravelGuide Android mobile application. This document shows how the TravelGuide android app will be designed, what architecture will be used, what components it will include of, what database structure will be used and how the UI will look like. This document will help developer, sample and any stake holder to know how system will work and will be developed for android phones.

1.2 Project Development Scope

TravelGuide is an android mobile application that is a grouping of Hotels, Restaurants, Transportation and Tourist Lures. TravelGuide permits customers to tourism plan and calculate the cost of traveling in any location using their Android cell phones. It has Offline obtainability using cached data & Real time approximation cost using Firebase Firestore. The System Modules are; Offline accessibility with cached data and Real time cost estimation with Firebase Firestore. System has a total of six modules:

- User Authentication Module
- Hotel Module
- Restaurant Module
- Transport Module
- Budget Estimation Module
- Admin Module

Android Application Frontend is to be created in React Native. Back-end will be written in Node and Express.js. Database will be MySQL and Firebase Firestore.

1.3 Definitions, Acronyms and Abbreviations.

- API - Application Programming Interface. Where Android application connects with the server.

- AsyncStorage - Key-value storage of the Android expedient which is persistent. Storing and caching JWT.
- BaaS - Backend as a Service. Firebase will be used for our BaaS solution. Landscapes include Firestore and Authentication.
- ERD -Entity Relationship Diagram.
- GPS - Global Positioning System. Enables the Android device to learn the nearby services via service discovery.
- GUI - Graphical User Interface.
- JWT - JSON Web Token. Stateless verification between the Android app and backend.
- RBAC - Role-Based Access Control. Used to restrict access to certain screens based on user role.
- React Native — Framework built on the creation of native Android applications in JavaScript.
- SDS - Software Design Specification

1.4 References

- [1] React Native official documentation. Available at: reactnative.dev
- [2] Node.js official documentation. Available at: nodejs.org
- [3] Firebase official documentation. Available at: firebase.google.com
- [4] MySQL official documentation. Available at: dev.mysql.com
- [5] React Native Maps documentation. Available at: github.com/react-native-maps/react-native-maps
- [6] Google Maps API documentation. Available at: developers.google.com/maps
- [7] S. Sharma and A. Gupta, "Design and development of a mobile-based travel planning system," IJCSIT, vol. 9, no. 3, pp. 45–49, 2018.

1.5 Overview of Document

Section 2. talks about the high-level Android scheme architecture and design justification.

Section 3. explains each constituent in further detail.

Section 4. goes over Android UI project and screens.

Sections 5. & 6. discuss recycle and design tradeoffs.

Section 7. provides pseudocode. Unit 8. has UML diagrams.

2. System Architecture Description

2.1 Section Overview

The following section plans the general 3-tier structure of TravelGuide Android application. It discusses the overall mobile constraints, data design, the program structure that illustrates how the three levels are arranged and architectural options that were taken into consideration.

2.2 General Constraints

- **Mobile Platform:** React Native should be used to create the frontend on the Android platform (API level 28 and above).
- **Stateless Authentication:** Secure admission is handled through JWT tokens which are stored in AsyncStorage on the Android device.
- **GPS Hardware:** The platform is based upon Android device GPS hardware to determine location-based services in actual time using React Native Maps.
- **Offline Support:** The app should allow offline basic functionality with the help of AsyncStorage and Firebase Firestore offline perseverance to explorers in low-connectivity regions of Pakistan.
- **Backend:** Node.js with express.js ends REST API endpoints to the Android app via HTTPS.
- **Database:** MySQL is used to stock all structured relational data. Firebase Firestore manages real-time budget syncing in the Android device.

Not all versions of No Budget have online payment, booking validation, and real-time price of operators.

2.3 Data Design

The system involves two database skills. All structured relational data are stored in MySQL. Firebase Firestore is used to tool real-time synchronization of live updates to the budget on the Trip Planning screen.

MySQL tables of the main database:

- users - user_id (PK), name, email, password_hash, role (Traveler, Hotel_Owner, Restaurant_Owner, Transport_Provider, Admin), created at.
- hotels - hotel id, user id (FK), name, city, location latitude, location longitude, room types, price per night, created at.
- restaurants — restaurant, user (FK), name, city, cuisine type, average meal value, created at.
- transport - transport_id, user_id (FK), vehicle-type, origin-city, destination-city, fare-per-person, departure-time, created-at.
- attractions attraction_id, name, city, category, entry fee, description, shaped at.
- tour plans -planid, userid (FK), destination city, travel dates, created at. budget - budget_id, plan_id (FK), hotel cost, food cost, transport cost, attraction cost, whole cost.

Firebase Firestore collections:

- trip_sessions — Real time session info of the trip currently under building. Keeps up to date as the traveler adds or eliminates services on the Trip Planning screen. The following ER diagram illustrates all the 7 tables and the relations:

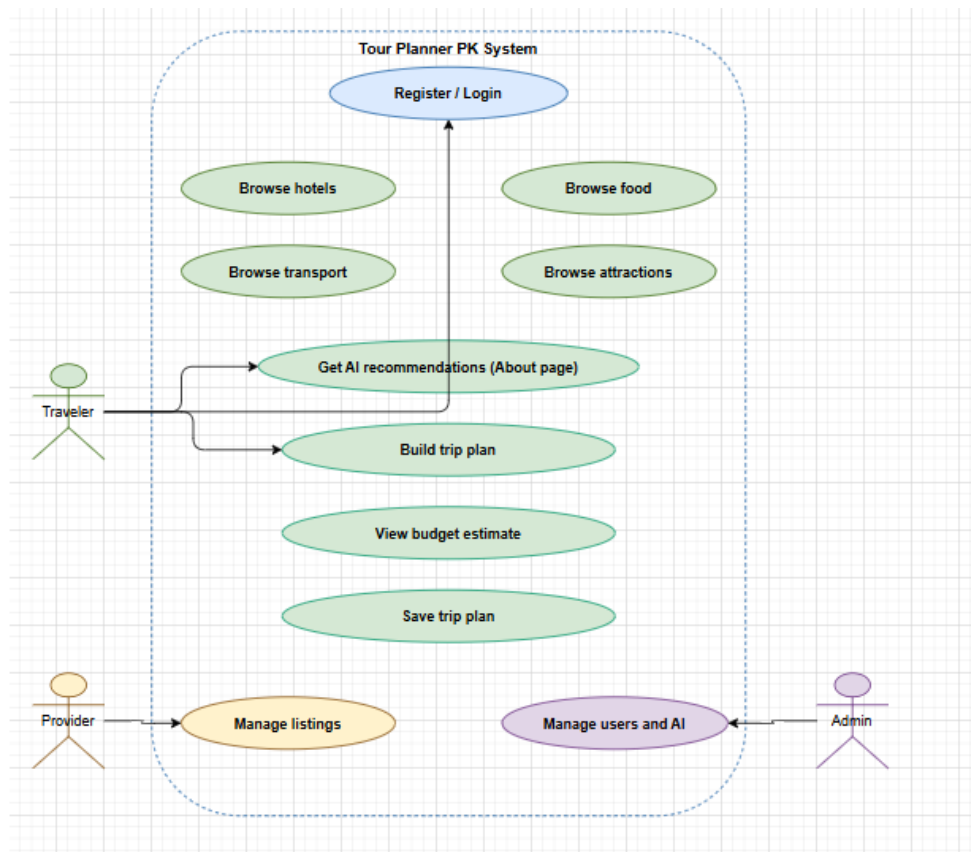


Figure 1: Entity Relationship Diagram — Database Schema

2.4 Program Structure

The TravelGuide Android app is founded on a 3-tier architecture:

Tier 1 - Presentation Layer (React Native Android App):

Android is a native stage of the mobile frontend. It will type all the screens, handle user interaction, make the HTTPS call to the backend REST API, and preserve a real-time Firebase Firestore auditor to update the budget in real-time. React Navigation is used to handle screen changes. It has the following main screens: the Login/Signup screen, the Traveler Home shade, the Provider Dashboard, the Hotels, Restaurants, Transport, Magnetisms, Trip Planning, Cost Summary, Map View, and Admin Panel.

Tier 2 -Application Layer (Node.js + Express.js Backend):

All business logic, JWT authentication, RBAC, input validation and database are full care of by the backend REST API. This backend is interconnected with through the Android app via HTTPS. Each endpoint that is protected must have a valid JWT token in the Authorization Carrier header.

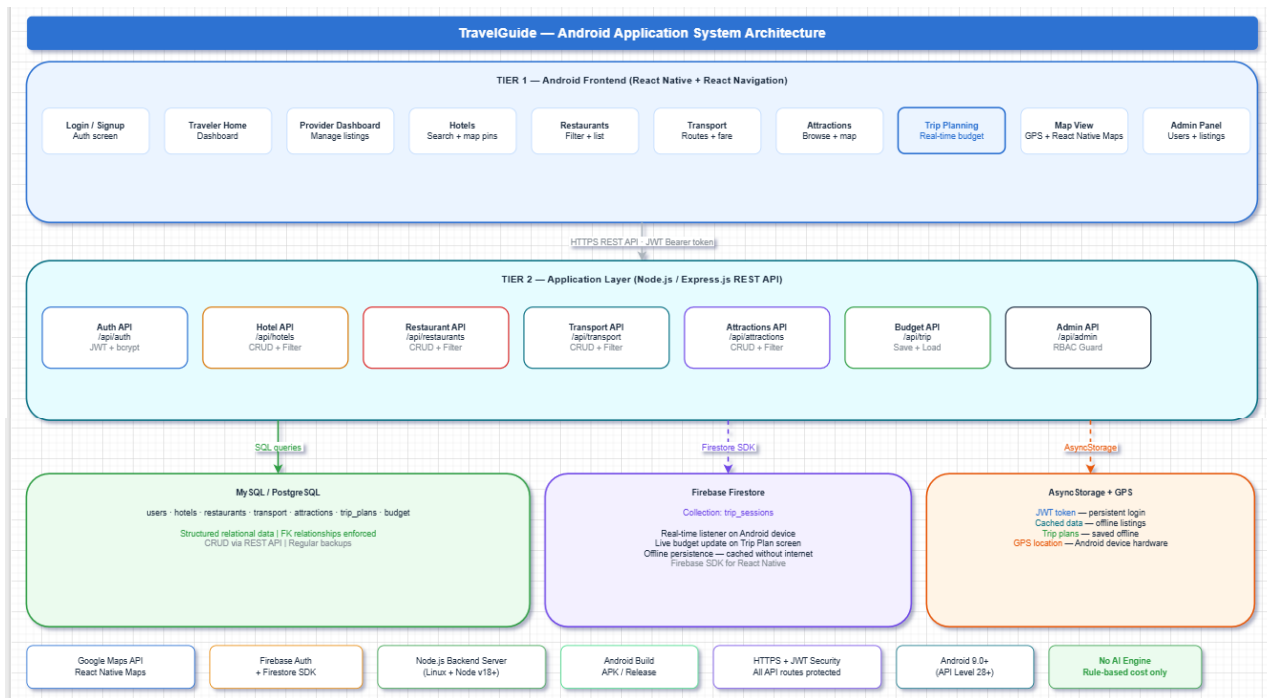
Tier 3 - Data Layer (MySQL + Firebase Firestore):

The structured interpersonal data are all stored in MySQL. Firebase Firestore delivers real-time updates of the consecutively trip plan session that runs the live budget display on the Android Trip Planning screen. The Firebase mobile SDK enables the application to keep Firestore spectators alive whenever the connectivity is poor.

2.5 Alternatives Considered

- Web application slightly than Android app: The original design was the web application using React.js. This was substituted by the reason that Android apps have a better and improved GPS integration, offline storage done AsyncStorage, native performance, and user experience in planning their travels in distant locations in Pakistan.
- Only database with Firebase Firestore: It was careful to use only Firestore. This was turned down then Firestore is a NoSQL document database which does not have complex interpersonal queries. MySQL is required to facilitate real filtering of hotels, restaurants and transport tables.
- Session-based authentication: Cookie-based meetings were taken into consideration. This was not accepted since JWT tokens are homeless and more suitable to a mobile REST API architecture.

2.6 System Architecture



3. Detailed Component description.

3.1 Section Overview

The following section will give an account of each of the seven elements of the TravelGuide android application in detail. These machineries are as follows: User Authentication, Hotel Module, Restaurant and Food Module, Transport Module, Tourist Magnetisms Module, Budget Estimation and Admin Module.

3.2 User Authentication Component

Identification: AuthComponent - backend API layer, which is appealed by the React Native Android app.

Type: Module - REST API supervisor using JWT middleware.

Purpose: Deals with registration, log in, and log out. Authenticates JWT tokens. Imposes RBAC through the system.

Function: Accepts registering requests of Android app. Use bcrypt to create a hash of password. Upon log in, authenticates password with bcrypt.compare and offers a signed JWT. The JWT is stored in the Android app and filled into the AsyncStorage and is devoted to each new API request.

Dependencies: MySQL users table to read and write records. This is dangerous to all other components since each safe API route must have a valid JWT.

Interfaces: POST /api/auth/register, POST /api/auth/login returns JWT to Android app. JWT is kept in AsyncStorage.

Processing: bcrypt round 10 salt. JWT expires 24 hours. A bad token results in 401 Unauthorized.

The sequence diagram below shows the complete login flow from the Android app to the backend:

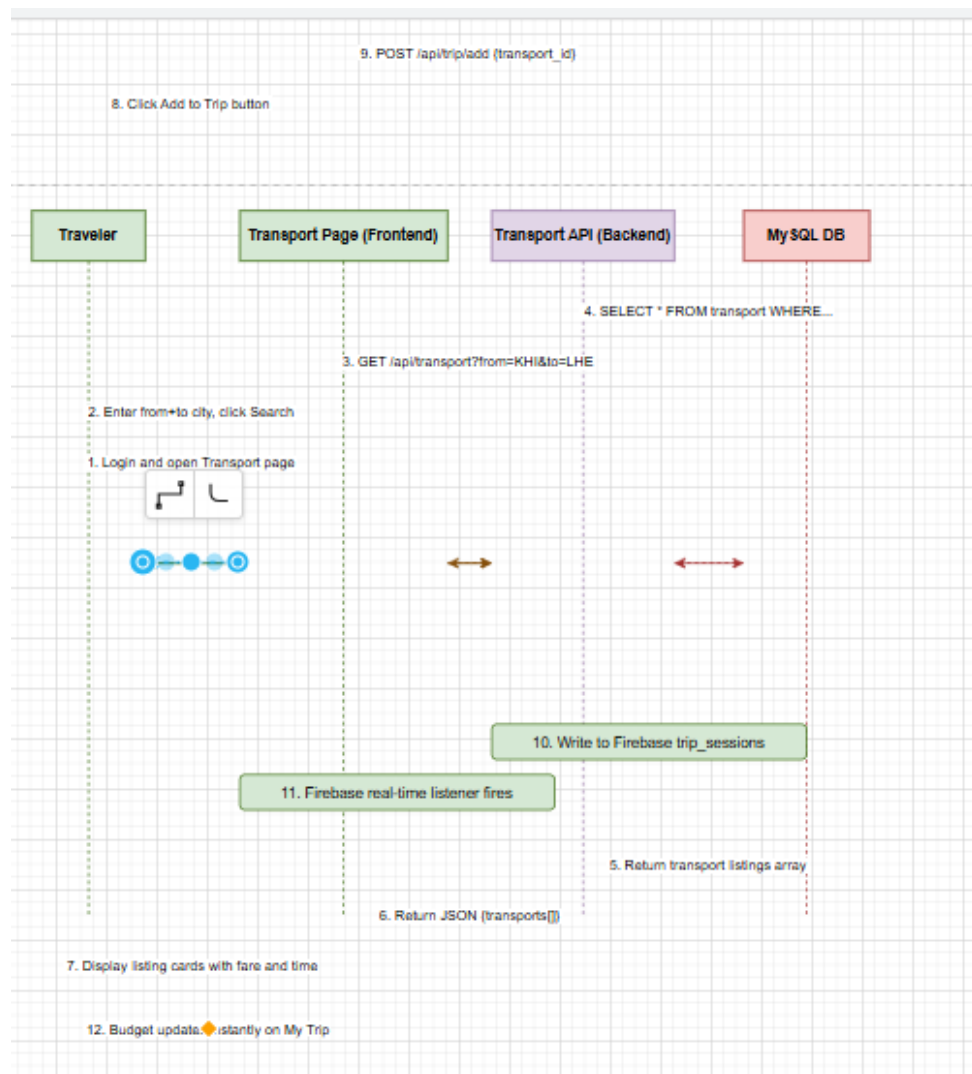


Figure 2: User Authentication — Login Sequence Diagram

3.3 Hotel Module Component

Identification: HotelModule - backend API layer, which is linked to the hotels board in the MySQL database.

Type: Module - REST API manager and CRUD operations.

Purpose: Enables owners of hotels to deal with Android-based schedules. Enables the explorers to search and browse hotels. Delivers pricing info to the budget estimation component.

Function: Hotel owners provide hotel information. Authenticates and stores in MySQL hotels table. Sifts finished listings to the travelers. GPS coordinates in the form of pins on React Native Maps.

Dependencies: AuthComponent to JWT. MySQL hotels table. Budget Approximation acquires pricing out of this module.

Interfaces: POST /api/hotels, GET /api/hotels, PUT /api/hotels/:id, DELETE /api/hotels/:id. Write roads require Hotel_Owner JWT.

Processing: Authenticates name, city, price_per_night present and positive. GPS coordinates validated within Pakistan boundaries. Only the proprietor can edit or delete their listing.

The use case diagram below shows all actions available to the Hotel Owner:

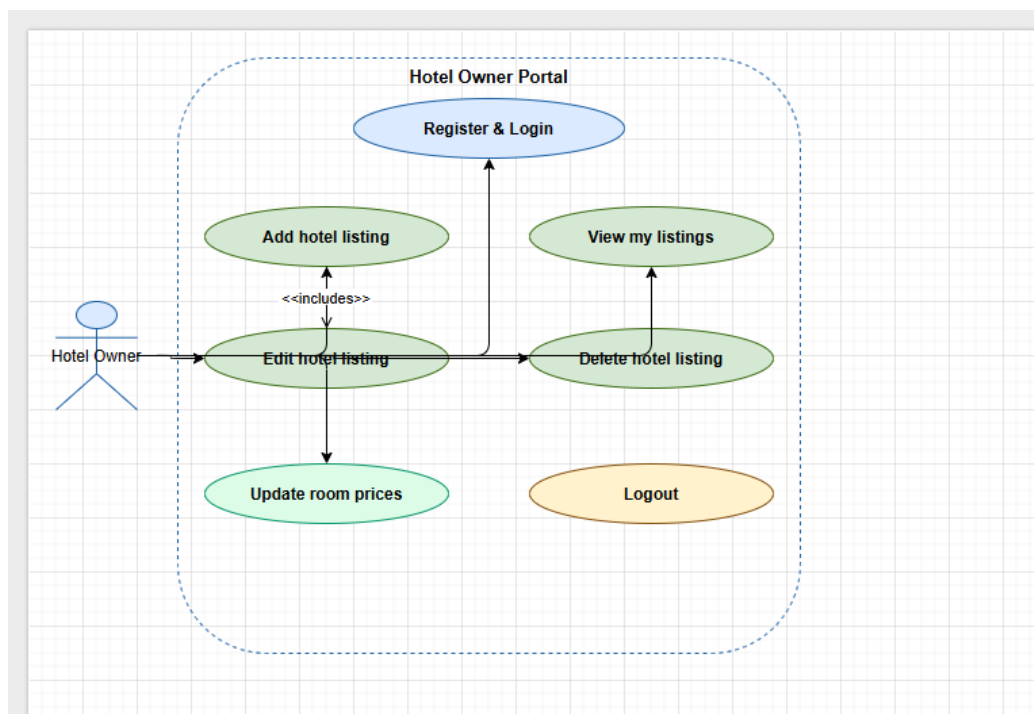


Figure 3: Hotel Module — Hotel Owner Use Case Diagram

The sequence diagram below shows the flow when a Hotel Owner adds a new hotel listing:

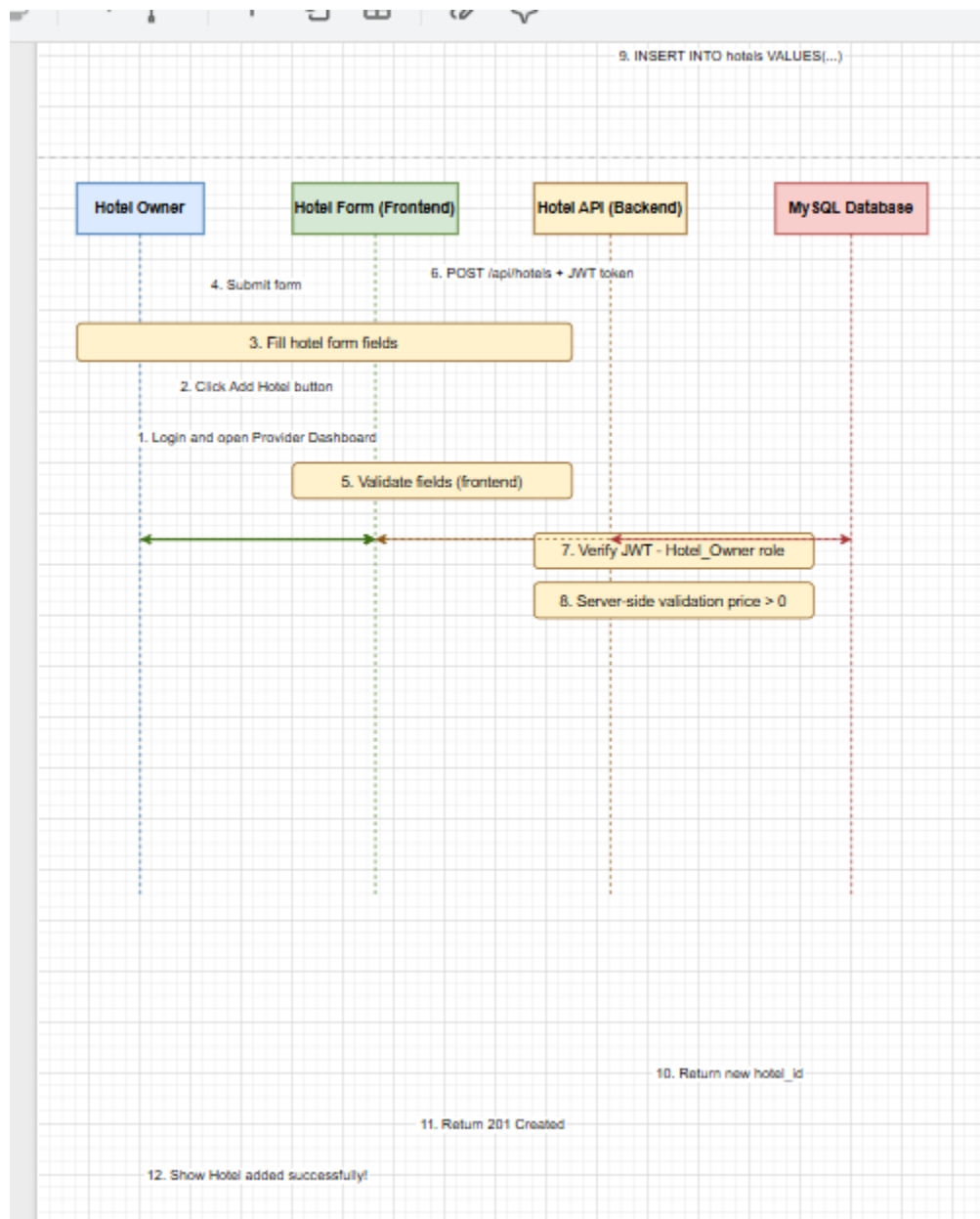


Figure 4: Hotel Module — Add Hotel Listing Sequence Diagram

3.4 Restaurant and Food Module Component

Identification: RestaurantModule — backend API layer, linked to the restaurants table in MySQL

Type: Module — REST API organizer with CRUD operations

Purpose: Allows restaurant owners to achieve listings from the Android app. Allows travelers to browse and filter restaurants. Offers meal pricing to the budget approximation component.

Function: Obtains restaurant data from owners. Stores and saves from MySQL restaurants table. Returns clean listings by city, cuisine type, and price range.

Dependencies: AuthComponent for JWT. MySQL restaurants table. Budget Estimate reads pricing from this module.

Interfaces: POST /api/restaurants, GET /api/restaurants, PUT /api/restaurants/:id, DELETE /api/restaurants/:id. Write ways require Restaurant_Owner JWT.

Processing: Validates average_meal_price is positive. One the owner can edit or delete their listing.

The use case diagram below shows all actions available to the Restaurant Owner:

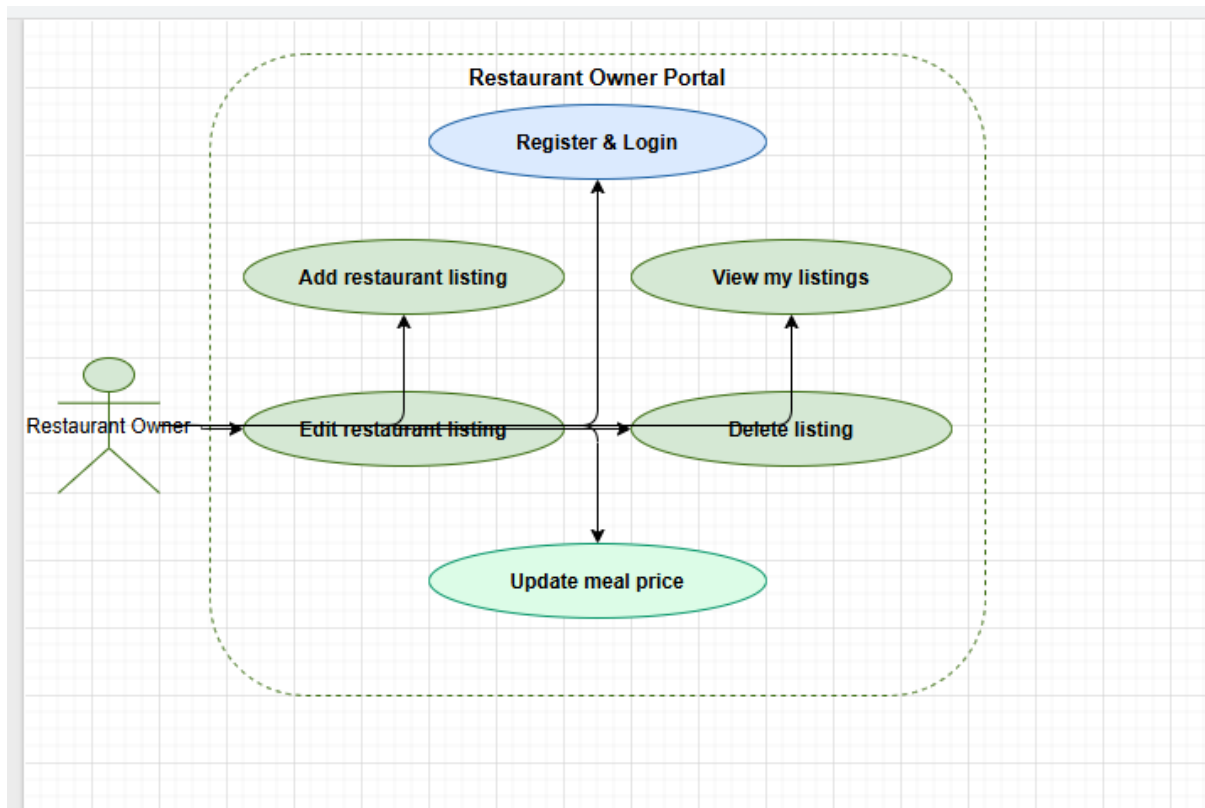


Figure 5: Restaurant Module — Restaurant Owner Use Case Diagram

The sequence diagram below shows the flow when a Restaurant Owner adds a new restaurant listing:

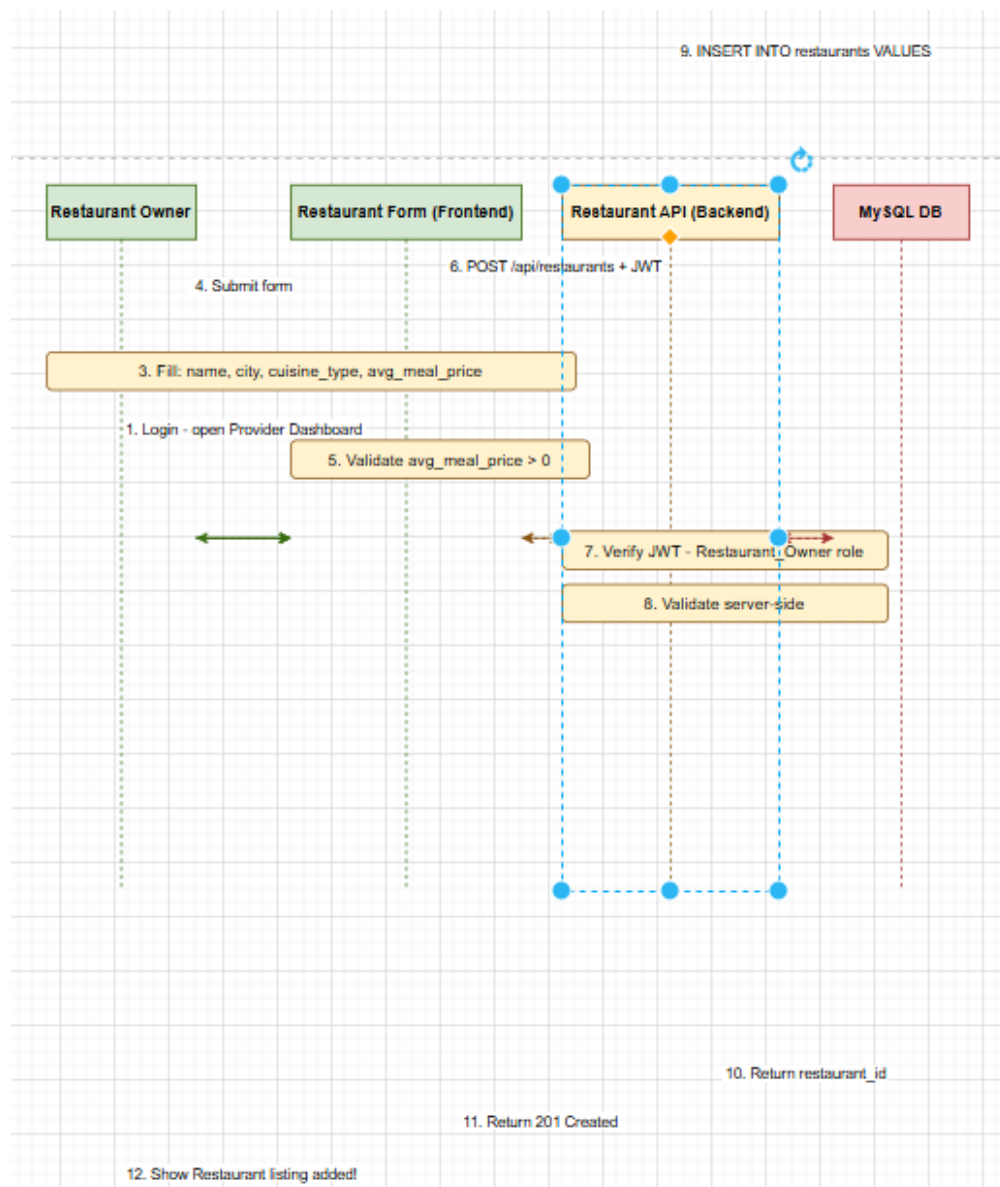


Figure 6: Restaurant Module — Add Restaurant Listing Sequence Diagram

3.5 Transport Module Component

Identification: TransportModule — backend API layer, connected to the transport table in MySQL

Type: Module — REST API controller with CRUD actions

Purpose: Allows transport providers to manage listings. Allows travelers to search carriage by route. Provides fare data to the budget estimation component.

Function: Receives transport data: vehicle type (Bus, Train, Car, Flight), route, source city, terminus city, fare per person. Returns filtered listings by route and vehicle type.

Dependencies: AuthComponent for JWT. MySQL transport table. Budget Estimate uses fare data.

Interfaces: POST /api/transport, GET /api/transport, PUT /api/transport/:id, DELETE /api/transport/:id. Write ways require Transport_Provider JWT.

Processing: Validates fare_per_person confident and cities present. Route distance fetched from Google Maps API or OpenStreetMap on Android.

The use case diagram below shows all actions available to the Transport Provider:

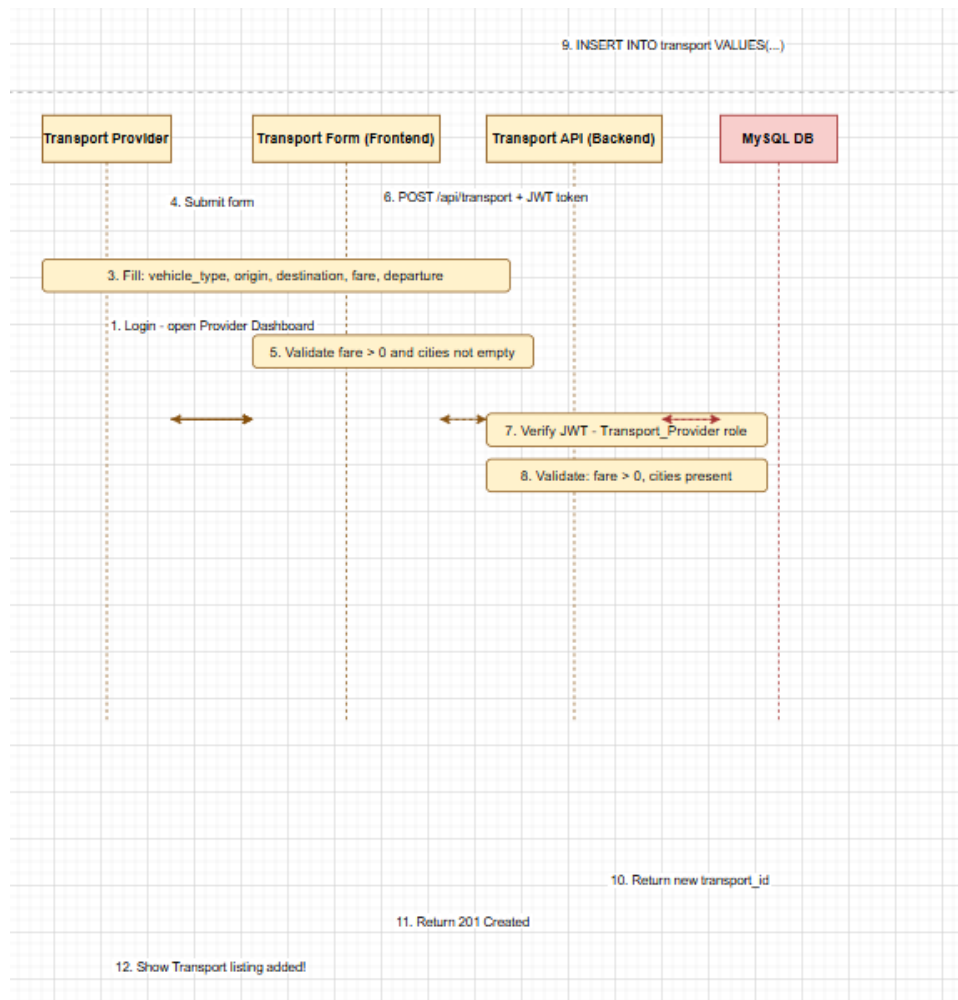


Figure 7: Transport Module — Transport Provider Use Case Diagram

The sequence diagram below shows the flow when a Transport Provider adds a listing and a Traveler searches and adds it:

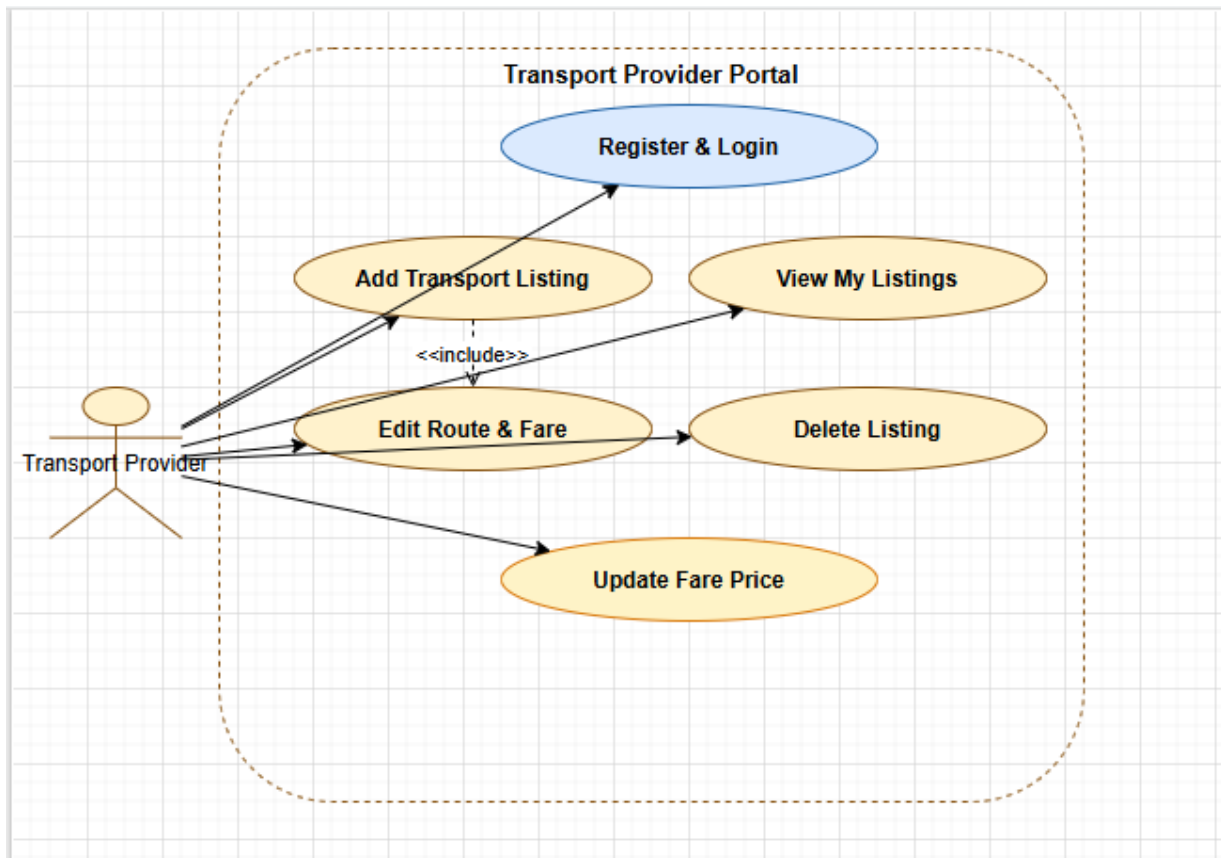


Figure 8: Transport Module — Add Transport & Traveler Search Sequence Diagram

3.6 Tourist Attractions Module Component

Identification: AttractionsModule — backend API layer, connected to the magnetisms table in MySQL

Type: Module — REST API organizer with CRUD operations

Purpose: Allows admin to manage tourist magnetism listings. Allows travelers to discover attractions by city and category. Provides entry fee data to the budget approximation component.

Function: Receives attraction data: name, city, GPS location, group, entry fee. Returns filtered listings by city and category. Pins shown on React Native Maps.

Dependencies: AuthComponent. MySQL attractions table. Budget Estimation delivers entry fees.

Interfaces: POST /api/attractions, GET /api/attractions, PUT /api/attractions/:id, DELETE /api/attractions/:id.

Processing: Entry fee authenticated as zero or positive. Group validated against: Historical, Religious, Nature, Adventure, Cultural.

3.7 Budget Estimation Component

Identification: BudgetEstimator - React Native Android front-end and Android backend API layer. Type: Android-side calculation mutual and Firebase Firestore real-time sync. Purpose: The purpose of this is to automatically compute the total trip cost as the tourist adds or removes services on the Trip Planning screen. Real time updates. Issues a warning when the total exceeds the budget limit stated. Function: The React Native application will write to Firebase Firestore trip_sessions when the tourist adds a service. The app has a Firestore real-time listener which directly re-calculates the total.

Formula: $\text{Total} = (\text{hotel price} \times \text{number of nights}) + (\text{meal price} \times \text{number of meals per day} \times \text{number of days}) + \text{transport fare} + \text{total of desirability entry fees}.$

Dependencies: Hotel, Restaurant, Transport, Attractions units to price on. React Native Firebase Firestore SDK. AuthComponent used to record session information.

Interface: Firebase Firestore trip_sessions collection. POST /api/ trip/ save in MySQL.

Processing: All arithmetic done in React Native JavaScript. Budgets alert displayed when the total is above the set limit. Written final plan to MySQL budget table.

3.8 Admin Module Component

Identification: AdminModule Backend API layer.

Type: Module - REST API controller and full access to platform organization.

Purpose: This module offers a central Android dashboard to the administrator to monitor stage statistics and handle all users and listings.

Dependencies: Implementation of AuthComponent with Admin role. Reads from and writes to all MySQL tables.

Interfaces: GET /api/admin/stats, GET /api/admin/users, PUT /api/admin/users/id, DELETE /api/admin/users/id, GET /api/admin/listings. All routes require Admin JWT.

Processing: The first middleware is Admin role check on all the routes of the administration. Non-admin requests send back 403 Forbidden.

4. User Interface Design

4.1 Section Overview

In this section, the Android UI design of the TravelGuide React Native application is defined. It includes the design guidelines, React Native GUI processes, and the description of screens.

4.2 Interface Design Rules

- **Mobile-First Design:** All the screens are Android smartphone-designed. Smallest touch target size is 48dp per Android Material Design guidelines.
- **Consistency:** The Bottom Navigation Bar, color scheme, font, and component style is common to all screens.
- **Real-Time Feedback:** Budget instantaneous updates on the Trip Planning screen whenever a service is added or not to the budget via Firebase Firestore listener.
- **Offline Indication:** When the device is not connected to the internet, a banner is displayed. Cached information is still available.
- **GPS Integration:** Map View display utilizes the Android device GPS to indicate the current position of the traveler and the pins of the services close to him.
- **Error Handling:** The validation errors are displayed in line with the corresponding input field immediately after the user enters an invalid form.

4.3 GUI Components

- **React Native Core:** View, Text, TextInput, TouchableOpacity, FlatList, ScrollView, ActivityIndicator, Modal, Alert of all typical Android UI components.
- **React Navigation:** Bottom Tab Navigator to triangulate the main screen and Stack Navigator to triangulate a drill-down flow.
- **React Native Maps:** Native Google Maps extension on Android with pins on the service, lines on the route and a GPS position.
- **Firebase Firestore SDK to React Native:** Live spectators of real-time budget changes on the Trip Planning screen.
- **AsyncStorage:** Android storage on the local device to store JWT token and hidden service listing to be used when the phone is offline.

- **React Native Vector Icons:** Icons of the lowest navigation bar and action buttons.

4.4 Detailed Description

Login / Signup Screen:

Enables one to create an account or sign in using his or her Android device. Fields emails, password, and role are covered. JWT token stored in AsyncStorage on fruitful login. User was taken to their role home page.

Traveler Home Screen:

Post-login main screen of travelers. Displays four cards of groups (Hotels, Restaurants, Transport, Attractions) and a Trip Planning button. Displays a rapid of the most recent saved trip plan.

Provider Dashboard Screen:

Hotel owners, Restaurant owners and Transport earners main screen. Displays their busy schedules in a FlatList with Add, Edit, and Delete gestures.

Hotels / Restaurants / Transport / Attractions Screens:

An apiece screen consists of a search bar with applicable filters on the top, React Native Maps opinion with facility location pins and a FlatList of service cards. All cards present the name of the ability, its location, cost, and Add to Trip button.

Trip Planning Screen:

Core traveler workflow display with services of choice and remove buttons and an automatically calculated budget display. Real-time budget updates through Firebase Firestore listener. Green notice indicates available budget. Red indicates that there is overage.

Cost Summary Screen:

Display on a read-only screen with a complete listed breakdown of the saved trip plan. Pre-tested to study and then verify the plan.

Map View Screen:

Full-screen React Native Maps view presentation chose the service locations as colored jots and GPS location of the traveler. Displays paths among trip stops. Hit a pin to display the price and facility name.

Admin Panel Screen:

Displays the number of users of the show, active listing and saved trip plans. Tabbed interface with Operators tab and Listings tab to manage the entire platform.

5. Reuse and Association with other Products.

- React Native: Wholesome Android frontend built on React Native. Single codebase does not require Android/iOS development.
- Express.js: REST API Backend framework based on Express.js that offers steering, middleware and request processing.
- Firebase SDK for React Native: Real-time budget informs and offline perseverance via Firebase Firestore mobile SDK.
- React Navigation: Standard React Native navigation library in case of Bottom Tab Navigator and Stack Navigator.
- React Native Maps: Native Google Maps on Android to pin service locations and GPS location.
- AsyncStorage: The stored data in AsyncStorage, the default React Native local storage, is hidden by JWT and offline.
- bcrypt: Hashing of passwords on the server using the bcrypt library.
- jsonwebtoken: JWT creation and verification with the npm library of jsonwebtoken.

There is no reuse of material used in previous semester projects. The design and code are completely original and constructed on the listed open-source libraries.

6. Design Decisions and Tradeoffs

Decision 1- Android Mobile App as opposed to Web Application:

The system is industrialized as a React Native Android app in its place of a web application. This has been determined since Android applications offer a better GPS hardware implementation, offline storage through the AsyncStorage, native performance, push notifications and user knowledge in planning the journey in the Pakistani mobile-first market place. The compromise is that web browsers cannot be used in Version 1.0.

Decision 2 Simple Module-Based Home Screen:

The Home screen shows four category cards (Hotels, Restaurants, Transport, Attractions) and has fresh module-based navigation. This leaves the awning tidy and makes first-time users get acquainted with the system quickly. The compromise is that users have to browse to separate modules.

Decision 3 - Role-Based Access Control (RBAC):

The RBAC Traveler, Facility Provider and Admin roles provide each user with a customized screen knowledge and the appropriate security. The tradeoff comes at the cost of greater development complexity to handle numerous screens and role checks.

Decision 4 - Offline Mode with Cached Data:

Offline tenacity with AsyncStorage and Firebase Firestore provides offline mode to enable explorers in the low-connectivity areas and Northern Pakistan, to access the data that was overloaded previously. The tradeoff is limited functionality down with no real-time updates.

Decision 5 Hybrid Database (MySQL + Firebase Firestore):

MySQL manages effective interpersonal queries among service tables. Firestore will supply real-time updates on the Android Trip Planning screen of the budget immediately without voting. Tradeoff is that the two databases upsurge the complexity.

Decision 6 — No Online Payment or Booking in Version 1.0:

Online reservation and payment are excluded from Version 1.0 to keep the project scope wielyd and avoid difficulties of payment security and financial APIs.

7. Pseudocode for Components

User Login Process (Android App to Backend):

```
FUNCTION loginUser(email, password):  
    user = database.query("SELECT * FROM users WHERE email = ?", email)  
    IF user not found THEN return 401 Unauthorized to Android app  
    isMatch = bcrypt.compare(password, user.password_hash)  
    IF isMatch is false THEN return 401 Unauthorized to Android app  
    token = jwt.sign({user_id, role}, SECRET_KEY, expires: 24h)
```

```
return 200 OK with token and user object  
Android app: AsyncStorage.setItem('jwt', token)
```

Budget Calculation Process (React Native):

```
FUNCTION calculateBudget(selectedServices, nights, mealsPerDay, days):  
    hotelCost = selectedHotel.price_per_night * nights  
    foodCost = selectedRestaurant.avg_meal_price * mealsPerDay * days  
    transportCost = selectedTransport.fare_per_person  
    attractionCost = SUM(selectedAttractions.entry_fee)  
    total = hotelCost + foodCost + transportCost + attractionCost  
    WRITE to Firebase Firestore trip_sessions (triggers real-time listener)  
    IF total > budget THEN show red Android notification  
    ELSE show green remaining budget notification
```

8. Appendices

8.1 Overall System Use Case Diagram

The general use case diagram below shows all five actors and their connections with the TravelGuide Android application. The Tourist is the main user who browses facilities and builds trip plans on their Android expedient. Service Providers manage their own listings. The Admin achieves all users and listings.

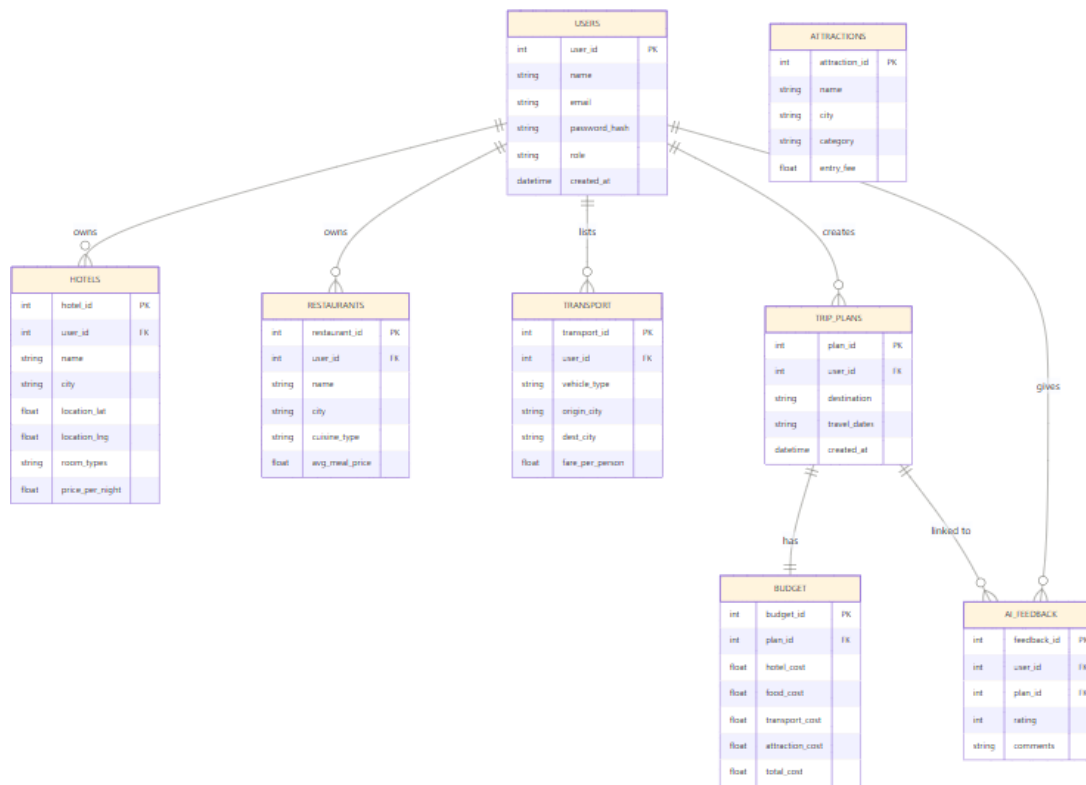


Figure 9: Overall System Use Case Diagram — All Actors and Use Cases

8.2 Entity Relationship Description

The main units: User has one role (Traveler, Hotel Owner, Restaurant Owner, Transport Provider, or Admin). A Hotel Owner User can have several Hotel listings. A Traveler User can have several TripPlans. Each TripPlan has exactly one Budget record. Each TripPlan positions selected services from Hotel, Restaurant, Transport, and Magnetism entities. The full ER Diagram is in Section 2.3 (Figure 1).

8.3 Sequence Diagram Reference

Login order is in Section 3.2 (Figure 2). Hotel Owner enhance listing sequence is in Section 3.3 (Figure 4). Restaurant Owner sequence is in Section 3.4 (Figure 6). Transport Provider and Traveler arrangement is in Section 3.5 (Figure 8).

8.4 Use Case Summary

Main custom cases: Register, Login (Android), Browse Hotels, Browse Restaurants, Browse Transport, Browse Attractions, Add Service to Trip Plan, View Budget Estimate, Except Trip Plan (offline accessible), View Map by GPS, Manage Users (Admin), Manage Listings (Admin). Overall use case diagram is in Section 8.1 (Figure 9).